

Building a 30M Parameter SQL Language Model

A Complete Architecture Guide

Every decision. Every option. Every reason why.

Model 30M Parameter SQL LM	Task Text-to-SQL Generation
Training Tokens 2 Billion	Hardware TPU v5e-1 (Free, Google Colab)
Framework JAX + Flax + Optax	Est. Training Time ~1–2 hours (full run)

1. What We Are Building and Why

Before we go component by component, it helps to understand the full picture: what we are building, why each major decision was made at a high level, and how all the pieces connect.

1.1 The Big Picture

A language model is a function that takes a sequence of tokens as input and outputs a probability distribution over the vocabulary for the next token. Do this repeatedly and you get text generation.

For our SQL model, the input looks like this:

Example Input / Output

Input: [Schema tokens] + [Question tokens] Output: [SQL query tokens]
Concretely: "CREATE TABLE orders (id INT, amount DECIMAL); How many orders over \$100?" → "SELECT COUNT(*) FROM orders WHERE amount > 100;"

Every architectural decision we make — how many layers, what kind of attention, which normalization — ultimately affects how well the model learns this mapping.

1.2 Why 30 Million Parameters?

Parameter count is the total number of learnable numbers in the model. More parameters = more capacity to memorize patterns, but also more compute, more memory, and more data needed.

Size	Capability	Free Compute Viability
1B+	Strong general language	Needs weeks even on TPU pods
100–500M	Good general language	Days on TPU v5e-1
30–100M	Good domain-specific tasks	Hours on TPU v5e-1 ✓
< 30M	Simple patterns only	Minutes but limited capability

✓ **OUR CHOICE:** 30M parameters. Large enough for real SQL understanding. Small enough to train fully in 1–2 hours on a free TPU, enabling rapid iteration.

1.3 Why SQL as the Target Task?

Training a general language model at 30M parameters is not competitive — SmolLM2-135M already dominates that space. But SQL is a narrow enough domain that a small, well-trained, specialized model can punch far above its weight.

- SQL has a restricted vocabulary — keywords like SELECT, FROM, WHERE are finite and predictable
- SQL has rigid structure — the grammar is well-defined, unlike free-form prose
- SQL success is objectively measurable — run the query and check if it produces the right result
- No great public sub-50M SQL model exists — you would own that benchmark space

1.4 The Full Architecture at a Glance

Here is how data flows through the model from raw text to predicted SQL:

Data Flow Through the Model

Step 1: Raw text → Tokenizer → Token IDs (integers)

Step 2: Token IDs → Embedding Lookup Table → Dense vectors

Step 3: Dense vectors → 16× Transformer Blocks → Contextual representations

Each Transformer Block contains:

(a) RMSNorm → Multi-Head Attention with RoPE → Residual Add

(b) RMSNorm → SwiGLU Feed-Forward Network → Residual Add

Step 4: Final representations → RMSNorm → LM Head (weight-tied) → Logits

Step 5: Logits → Softmax → Probability over vocabulary → Predicted next token

We will now go through each of these steps in detail, covering every design decision, all available options, and exactly why we chose what we chose.

2. Tokenization

Tokenization is the process of converting raw text into a sequence of integers (token IDs) that the model can process. This is the very first step in the pipeline and one of the most consequential decisions you will make.

2.1 Why Not Just Use Characters or Words?

The two most obvious approaches — splitting by character or by word — both have fatal flaws at scale.

Approach	Example: 'SELECT'	Fatal Problem
Character-level	S, E, L, E, C, T (6 tokens)	Sequences become very long. Attention is $O(n^2)$ so 6× more tokens = 36× more compute.
Word-level	SELECT (1 token)	Vocabulary explodes to millions. 'unhappiness', 'unhappier', 'unhappily' are three separate tokens. Cannot handle new words.
Subword (BPE) ✓	SELECT (1 token usually)	Best of both worlds. Handles rare words by splitting them, keeps common words whole.

2.2 Byte Pair Encoding (BPE) — How It Works

BPE starts with individual bytes (256 possible values — covers every possible character in any language) and iteratively merges the most frequent adjacent pairs.

BPE Training Process (Simplified)

Start: S E L E C T F R O M (individual characters)

Step 1: 'EC' appears 10,000 times → merge → S 'EC' T F R O M

Step 2: 'SE' appears 8,000 times → merge → 'SE' 'EC' T F R O M

Step 3: 'FR' appears 7,500 times → merge → 'SE' 'EC' T 'FR' O M

...repeat until vocabulary is full...

Result: Common SQL keywords like SELECT, FROM, WHERE become single tokens.

2.3 Vocabulary Size Decision

Vocabulary size is how many unique tokens exist. This is a critical hyperparameter because the embedding table size = vocab_size × hidden_dim.

Vocab Size	Used By	Embedding Params (hidden=384)	Our Situation
32,768	LLaMA 2, SmolLM2	$32768 \times 384 = 12.6\text{M}$	Too large — 42% of our entire model just for embeddings
16,384	Various small models	$16384 \times 384 = 6.3\text{M}$	Better but still 21%
8,192 ✓	Our choice	$8192 \times 384 = 3.1\text{M}$	Only 10% of model. Frees 9.5M params for transformer layers.
4,096	Extreme compression	$4096 \times 384 = 1.6\text{M}$	May not cover all SQL patterns and English words reliably

The key insight: SQL has a very restricted vocabulary. You do not need 32,000 tokens to represent SELECT, FROM, WHERE, JOIN, GROUP BY, ORDER BY, and common English question words. 8,192 tokens covers everything you need with room to spare.

✓ **OUR CHOICE:** 8,192 vocabulary size. BPE tokenizer trained on SQL files, StackOverflow SQL posts, and a small English sample. This keeps the embedding table small (3.1M params) freeing capacity for actual reasoning layers.

2.4 Should We Train Our Own or Reuse an Existing Tokenizer?

Option	Pros	Cons
Train our own on SQL data	SQL keywords tokenize as single tokens. Better fertility (fewer tokens per word). Domain optimized.	Extra work. Need tokenizer training code.
Reuse SmolLM2/LLaMA tokenizer	Zero work. Well tested.	32k vocab (too large for us). SQL keywords may be split. Wasteful for our domain.

✓ **OUR CHOICE:** Train our own BPE tokenizer on ~100M characters of SQL + English data. Use HuggingFace tokenizers library. Verify that SELECT, FROM, WHERE, JOIN, GROUP BY, ORDER BY, HAVING, INSERT, UPDATE, DELETE all tokenize as single tokens — this matters for model efficiency.

2.5 Special Tokens

Special tokens are reserved IDs that mark structural boundaries in the text. Every tokenizer needs them.

Token	Meaning	Example Use
<bos>	Beginning of Sequence	Added at the start of every training example
<eos>	End of Sequence	Added at the end. Model learns to predict this when SQL is complete.
<pad>	Padding	Used to pad shorter sequences to equal length (less needed with packing)
<unk>	Unknown	Fallback for characters not in vocabulary. Rare with byte-level BPE.

2.6 Fertility — The Tokenizer Quality Metric

Fertility measures the average number of tokens per word. Lower is better because fewer tokens = shorter sequences = faster training.

Fertility Examples

Good tokenizer on SQL: 'SELECT COUNT(*) FROM orders' → 6 tokens (fertility ≈ 1.2)

Bad tokenizer on SQL: 'SELECT COUNT(*) FROM orders' → 12 tokens (fertility ≈ 2.4)

Target: < 1.4 tokens per SQL keyword on average.

Test: tokenize 1,000 common SQL queries. Count total tokens $\div$ total words.

3. Token Embeddings

After tokenization, each token ID needs to be converted into a dense vector — a list of floating point numbers that the model can do math with. This conversion is called embedding.

3.1 What an Embedding Is

The embedding table is a simple lookup table: a matrix of shape [vocab_size × hidden_dim]. Token ID 4821 → look up row 4821 → get a 384-dimensional vector. That is it. The magic is that these vectors are learned during training and gradually encode semantic meaning.

Intuition

After training, similar tokens end up with similar vectors. The model learns that SELECT and CHOOSE should have similar representations because they appear in similar contexts. It learns that INT and VARCHAR are both data types. These relationships are not programmed — they emerge from seeing billions of examples.

3.2 Hidden Dimension — The Most Important Hyperparameter

The hidden dimension (also called d_model or embed_dim) is how many numbers are in each token's vector representation. It is the core width of the entire model — every layer operates at this dimension.

Hidden Dim	Roughly Used By	Total Params at 16 Layers
256	Ultra-tiny models	~8–12M params
384 ✓	Our choice	~28–32M params
512	SmolLM2-135M range	~50–60M params
768	GPT-2 Small	~125M params

384 hits the sweet spot for 30M total parameters. Going wider would push us over our budget. Going narrower would leave too little representational capacity per token.

✓ **OUR CHOICE:** Hidden dimension = 384. This flows through the entire model — every attention layer, every MLP, every normalization layer operates on 384-dimensional vectors.

3.3 Weight Tying — A Free Lunch

The model has two places that relate tokens to vectors: the input embedding table (token ID → vector) and the output LM head (vector → logit scores over vocabulary). These are both [vocab_size × hidden_dim] matrices — the same shape.

Weight tying means sharing the same matrix for both. The input embedding and output projection are literally the same weights.

Approach	Parameters Used	Quality Effect
Separate weights (no tying)	$2 \times (8192 \times 384) = 6.3\text{M}$ params	Slightly more flexible
Weight tying ✓	$1 \times (8192 \times 384) = 3.1\text{M}$ params	Saves 3.1M params with minimal quality loss. Often slightly better.

✓ **OUR CHOICE:** Weight tying enabled. Saves 3.1M parameters (about 10% of our entire model) with essentially no downside. Used by LLaMA 2, SmolLM2, GPT-2, and most small models.

3.4 Embedding Initialization

How should the embedding matrix be initialized before training? This matters more than people realize.

Initialization	Effect
Default PyTorch (std=1.0)	Embeddings start large → large initial logits → unstable early training. Bad.
Small std (0.02) ✓	Embeddings start near zero → logits start near uniform → stable training from step 1.
Uniform random	Similar to above, acceptable.

✓ **OUR CHOICE:** Initialize embedding table with standard deviation 0.02. This is a standard practice in all modern language models and prevents training instability in the early steps.

4. Positional Embeddings

The transformer attention mechanism has no built-in sense of order. Without positional information, 'dog bites man' and 'man bites dog' would produce identical representations. Positional embeddings solve this by encoding where each token appears in the sequence.

4.1 The Four Main Approaches

Option A: Learned Absolute Positional Embeddings (GPT-2 style)

Add a separate learned vector for each position. Position 0 gets vector_0, position 1 gets vector_1, and so on. These vectors are learned during training just like token embeddings.

Aspect	Detail
How it works	$\text{output} = \text{token_embedding} + \text{position_embedding}[\text{position_index}]$
Parameters	$\text{context_length} \times \text{hidden_dim}$ extra parameters
Problem 1	Cannot generalize beyond training length. If trained on 512, position 513 is meaningless.
Problem 2	Absolute position is a weak signal. 'The' at position 5 and 'The' at position 50 should behave similarly but get different embeddings.
Used by	GPT-2, GPT-3 (with modifications), BERT

Option B: Sinusoidal Absolute Embeddings (Original Transformer)

Instead of learned vectors, use fixed sine and cosine functions at different frequencies to encode position. No parameters needed.

Aspect	Detail
How it works	position_i is encoded as sin/cos waves at frequencies that decrease with dimension index
Advantage	No extra parameters. Can extrapolate to longer sequences somewhat.
Problem	Still absolute — same weakness as learned absolute. Poor relative position awareness.
Used by	Original 'Attention is All You Need' paper (2017). Rarely used now.

Option C: ALiBi — Attention with Linear Biases

Instead of modifying embeddings, ALiBi adds a penalty to attention scores that increases linearly with distance between tokens. Tokens far apart attend to each other less.

Aspect	Detail
How it works	$\text{attention_score}(i,j) = QK^T / \sqrt{d} - \text{slope} \times i - j $
Advantage	Good length generalization. Zero extra parameters.
Problem	Does not encode absolute position at all. Works for most tasks but can hurt on tasks where absolute position matters.
Used by	MPT, BLOOM, some older models

Option D: RoPE — Rotary Position Embeddings (Our Choice)

RoPE is the most elegant and widely adopted approach. Instead of adding position information to the embedding vectors, it rotates the Query and Key vectors inside the attention mechanism by an angle that depends on position.

The mathematical beauty: when you compute the dot product between a query at position m and a key at position n , the rotation naturally produces a result that depends only on the relative distance $(m - n)$. The model learns to use relative position, not absolute.

RoPE Intuition

Think of each dimension pair in your query/key vector as a clock hand.

Position 0 = 12 o'clock.

Each step forward rotates the hand by a fixed angle.

Low-frequency dimensions rotate slowly → encode long-range position.

High-frequency dimensions rotate fast → encode short-range position.

When you compute $Q \cdot K^T$, you're measuring how 'in sync' two positions are.

Nearby tokens = similar angles = high attention score.

Distant tokens = different angles = lower attention score.

This happens automatically from the math — not from explicit distance penalties.

Approach	Relative Position Aware?	Extra Parameters?	Beyond-training Extrapolation?	Modern Usage
Learned Absolute	No	Yes (small)	No	Obsolete
Sinusoidal	No	No	Partial	Obsolete
ALiBi	Yes (via penalty)	No	Good	Declining
RoPE ✓	Yes (via rotation)	No	Good with tricks	Dominant

4.2 RoPE Base Frequency

RoPE has one key hyperparameter: the base frequency, which controls how fast the rotation happens. Higher base = slower rotation = better ability to distinguish long-range positions.

Base	Used By	Good For
10,000	Original RoPE paper, LLaMA 1	Context lengths up to ~2,048
100,000	Mistral, CodeLlama	Context lengths up to ~8,192
500,000	LLaMA 3	Very long contexts, also better generalization at short contexts

✓ **OUR CHOICE:** RoPE positional embeddings with base = 10,000. Our context length is 512 tokens (SQL queries and schemas fit easily). At this length, base 10,000 is perfectly sufficient. No extra parameters. Relative position awareness out of the box.

5. The Attention Mechanism

Attention is the core computation of the transformer. It allows every token to look at every other token and decide which ones are relevant to understanding the current token's meaning. This is what makes transformers so powerful — full context awareness at every layer.

5.1 Scaled Dot-Product Attention — The Foundation

Before we get to multi-head and grouped-query variants, understand the base operation.

How Single-Head Attention Works

Given a sequence of token vectors X (shape: $[\text{seq_len} \times \text{hidden_dim}]$):

1. Project X into three matrices:

Query (Q) = $X \times W_Q$ 'What am I looking for?'

Key (K) = $X \times W_K$ 'What do I contain?'

Value (V) = $X \times W_V$ 'What will I give if selected?'

2. Compute attention scores:

$\text{scores} = Q \times K^T / \sqrt{\text{head_dim}}$ (shape: $[\text{seq_len} \times \text{seq_len}]$)

The division by $\sqrt{\text{head_dim}}$ prevents the dot products from getting too large, which would push softmax into saturation (near-zero gradients).

3. Apply causal mask (for language modeling):

Each token can only attend to itself and past tokens — not future tokens.

Future positions get score = -infinity so they become 0 after softmax.

4. Softmax to get attention weights (each row sums to 1):

$\text{weights} = \text{softmax}(\text{masked_scores})$

5. Weighted sum of values:

$\text{output} = \text{weights} \times V$

6. Project back to hidden_dim:

$\text{final} = \text{output} \times W_O$

5.2 Why Scale by $\sqrt{\text{head_dim}}$?

When head_dim is large (e.g. 64), the dot products $Q \cdot K$ can become very large in magnitude. If you feed large numbers into softmax, the gradient nearly vanishes — the model stops learning. Dividing by $\sqrt{64} = 8$ keeps the scores in a range where softmax still has useful gradients. This is a small detail that matters enormously in practice.

5.3 Multi-Head Attention — Why Multiple Heads?

A single attention head can only learn one kind of relationship between tokens. Multi-head attention runs several attention operations in parallel, each with smaller head_dim, then concatenates the results.

Intuition for Multiple Heads

Head 1 might learn: 'which table does this column belong to?' Head 2 might learn: 'which tokens are SQL keywords?' Head 3 might learn: 'what is the subject of this English question?' Head 4 might learn: 'which tokens are numeric values or conditions?' No head is explicitly programmed to do this — these patterns emerge from training. Multiple heads allow the model to simultaneously track multiple types of relationships.

The total compute stays the same: 6 heads × 64 head_dim = 384 = same as 1 head × 384. But the representations are richer because multiple independent attention patterns are computed.

5.4 Grouped Query Attention (GQA) — Should We Use It?

In standard Multi-Head Attention (MHA), every head has its own Q, K, and V projection. In GQA, multiple query heads share a single K and V pair.

Variant	Structure	Best For
MHA (Multi-Head)	6 Q heads, 6 K heads, 6 V heads	Training — maximum expressiveness
GQA (Grouped Query)	6 Q heads, 2 K heads, 2 V heads (3:1 ratio)	Inference at larger scales — smaller KV cache
MQA (Multi-Query)	6 Q heads, 1 K head, 1 V head	Maximum inference speed, some quality loss

✓ **OUR CHOICE:** Standard Multi-Head Attention (MHA) — no GQA. At 30M parameters with 512 context length, the KV cache is tiny (a few megabytes). GQA saves memory primarily when you have many layers, large hidden dims, and long contexts. The complexity cost outweighs the benefit at our scale.

5.5 Our Attention Configuration

30M Model Attention Dimensions

hidden_dim: 384

num_heads: 6
head_dim: $384 \div 6 = 64$

Q projection: $384 \times (6 \times 64) = 384 \times 384 = 147,456$ parameters

K projection: $384 \times (6 \times 64) = 384 \times 384 = 147,456$ parameters

V projection: $384 \times (6 \times 64) = 384 \times 384 = 147,456$ parameters

O projection: $384 \times 384 = 147,456$ parameters

Total per layer: ~590,000 parameters

head_dim = 64 is a standard, well-tested value. Powers of 2 are preferred for hardware efficiency.

5.6 Flash Attention — Algorithmic Optimization

Flash Attention is not a different attention mechanism. It computes the exact same mathematical result as standard attention, but does so using a smarter algorithm that avoids writing the full attention matrix to GPU/TPU memory.

Aspect	Standard Attention	Flash Attention
Memory usage	$O(n^2)$ — stores the full [seq_len × seq_len] matrix	$O(n)$ — computes in tiles, never stores full matrix
Speed	Memory bandwidth bound at long sequences	2–4× faster in practice
Result	Identical	Identical (same math, different execution order)
Implementation	Built into PyTorch / JAX	Separate library (flash-attn), or JAX pallas kernels

✓ **OUR CHOICE:** Use Flash Attention if available on TPU v5e via JAX/Pallas. At our sequence length of 512, the benefit is modest but still real. The key point: it is a free speedup with zero quality impact.

6. Normalization Layers

Normalization layers stabilize training by keeping activations in a reasonable numerical range. Without them, values in deep networks tend to grow or shrink exponentially, making training unstable or impossible.

6.1 The Problem Normalization Solves

Why Normalization Matters

In a 16-layer network, each layer's output feeds into the next. If layer 1 amplifies values by $1.1\times$ on average, after 16 layers values have grown by $1.1^{16} = 4.6\times$. If it amplifies by $1.5\times$, after 16 layers values have grown by $1.5^{16} = 656\times$. These exploding values cause NaN losses, zero gradients, and failed training. Normalization keeps values bounded at each layer.

6.2 The Main Options

Option A: Layer Normalization (LayerNorm)

Normalizes across the feature dimension (hidden_dim) for each token independently. Introduced in 'Attention is All You Need'.

LayerNorm Formula

For a vector x of dimension d :

mean = average of $x[0], x[1], \dots, x[d-1]$

var = variance of x

output = $(x - \text{mean}) / \sqrt{\text{var} + \text{epsilon}} * \gamma + \beta$

γ and β are learned parameters (one per dimension) that allow the layer to scale and shift the normalized values.

Option B: RMSNorm — Root Mean Square Normalization (Our Choice)

RMSNorm drops the mean subtraction step. It normalizes purely by the root mean square of the values. Simpler, faster, and empirically equivalent or better.

RMSNorm Formula

For a vector x of dimension d :

rms = $\sqrt{\text{mean of } x^2}$

output = $x / \text{rms} * \gamma$

No beta parameter (no mean subtraction = no shift needed).
~15% faster than LayerNorm. Used by LLaMA, Qwen, Gemma, and most modern models.

Norm Type	Parameters	Speed	Used By
LayerNorm	gamma + beta (2 × hidden_dim)	Baseline	BERT, GPT-2, older models
RMSNorm ✓	gamma only (1 × hidden_dim)	~15% faster	LLaMA, Gemma, Qwen, SmolLM2, us
GroupNorm	Similar to LayerNorm	Variable	Mostly used in vision, not LLMs

6.3 Pre-Norm vs Post-Norm

Where you place the normalization layer relative to the attention/MLP computation matters a lot for training stability.

Placement	Order	Effect
Post-Norm (original transformer)	Attention → Add residual → Normalize	Can be unstable at large depths. Requires careful LR warmup.
Pre-Norm ✓ (modern standard)	Normalize → Attention → Add residual	Much more stable training. Can use higher learning rates. Used by all modern LLMs.

✓ **OUR CHOICE:** RMSNorm with Pre-Norm placement. Applied before both the attention block and the MLP block in each transformer layer. This is the architecture used by LLaMA, Qwen, Gemma, and SmolLM2. Empirically more stable than post-norm, especially at smaller scales where training dynamics are more fragile.

7. Feed-Forward Networks (MLP Block)

Each transformer layer has two sub-components: the attention block and the feed-forward network (also called the MLP block). The attention block is about relationships between tokens. The MLP block is about processing each token independently after those relationships are captured.

7.1 What the MLP Block Does

Intuition

If attention answers 'which other tokens are relevant to me?', the MLP block answers 'given what I know about my context, what should my new representation be?' It is a per-token nonlinear transformation that lets the model compute complex functions of the attended information.

7.2 The Options for Activation Functions

The nonlinearity in the MLP block is what gives the network its expressive power. Without it, a stack of linear layers would just be one linear layer.

Option A: ReLU (Rectified Linear Unit)

The classic. Outputs 0 for negative inputs, x for positive inputs.

Aspect	Detail
Formula	$\text{ReLU}(x) = \max(0, x)$
Problem	'Dying ReLU' — neurons that always output 0 stop learning. Also sharp kink at 0 can cause issues.
Used by	Early transformers, Feed-Forward Networks in original paper

Option B: GELU (Gaussian Error Linear Unit)

A smooth approximation to ReLU. Used in BERT, GPT-2. Better than ReLU in practice.

Aspect	Detail
Formula	$\text{GELU}(x) \approx 0.5x(1 + \tanh(\sqrt{2/\pi}(x + 0.044715x^3)))$
Advantage	Smooth curve, no dying neuron problem
Used by	BERT, GPT-2, GPT-3, older models

Option C: SwiGLU — Swish-Gated Linear Unit (Our Choice)

SwiGLU uses a gating mechanism. Instead of one up-projection, it has two: one goes through Swish activation, the other is linear. Their element-wise product is the gating operation.

SwiGLU Formula

Standard MLP: $\text{output} = W_{\text{down}}(\text{activation}(W_{\text{up}}(x)))$

SwiGLU: $\text{output} = W_{\text{down}}(\text{SiLU}(W_{\text{gate}}(x)) * W_{\text{up}}(x))$

Where $\text{SiLU}(x) = x * \text{sigmoid}(x)$ (also called Swish)

The gate controls how much of $W_{\text{up}}(x)$ flows through.

This selective gating gives the model more fine-grained control over information flow.

Activation	Parameters for same capacity	Quality	Used By
ReLU	2 matrices (up + down)	Baseline	Old models
GELU	2 matrices (up + down)	+2–3% vs ReLU	GPT-2, BERT
SwiGLU ✓	3 matrices (gate + up + down)	+4–5% vs ReLU	LLaMA, Qwen, Gemma, PaLM

Note: SwiGLU needs 3 matrices instead of 2, which means the `intermediate_dim` should be 2/3 of what you would use with standard GELU to keep parameter count equal. The LLaMA family uses approximately $2.67 \times \text{hidden_dim}$ as `intermediate_dim` (after the 2/3 adjustment).

7.3 MLP Expansion Ratio

The `intermediate_dim` controls how wide the MLP expands before contracting back. This is the expansion ratio.

Expansion Ratio	<code>intermediate_dim</code> (<code>hidden=384</code>)	Parameters Per Layer
2×	768	$2 \times (384 \times 768) + (768 \times 384) \approx 0.9\text{M}$
2.67× ✓ (SwiGLU adjusted)	896	~1.0M per layer
4×	1,536	2M per layer — too large for 30M model

✓ **OUR CHOICE:** SwiGLU activation with `intermediate_dim = 896` (approximately $2.34 \times \text{hidden_dim}$). This gives the quality benefits of SwiGLU within our parameter budget. The gate + up + down structure allows information-selective processing that outperforms simpler activations.

8. Residual Connections

Residual connections (also called skip connections) are one of the most important innovations in deep learning. They are elegantly simple but solve a fundamental problem.

8.1 The Problem: Vanishing Gradients in Deep Networks

When training a deep neural network, gradients (the signals that tell weights how to update) flow backward through layers via the chain rule. In a 16-layer network, the gradient at layer 1 is the product of 16 Jacobian matrices. If each is slightly smaller than 1, the gradient at layer 1 is 1^{16} = near zero. The early layers stop learning. This is the vanishing gradient problem.

8.2 The Solution: Add the Input to the Output

Residual Connection Formula

Without residual: $\text{output} = \text{layer}(x)$

With residual: $\text{output} = x + \text{layer}(x)$

The model now only needs to learn the 'residual' (the change), not the entire transformation.

Gradient of x through this block:

$$d(\text{output})/d(x) = 1 + d(\text{layer}(x))/d(x)$$

The '1' guarantees gradient always has a path back — no matter how small $d(\text{layer})/d(x)$ becomes, the gradient is at least 1.

This makes deep networks (16, 24, 32 layers) trainable.

8.3 How It Appears in Our Architecture

Each transformer block applies two residual connections: one around the attention sub-layer, one around the MLP sub-layer.

Full Transformer Block Forward Pass

Input: x (shape: [batch, seq_len, 384])

Attention sub-layer with residual

$x = x + \text{Attention}(\text{RMSNorm}(x))$

MLP sub-layer with residual

$x = x + \text{MLP}(\text{RMSNorm}(x))$

Output: x (same shape: $[\text{batch}, \text{seq_len}, 384]$)

Notice: Pre-Norm is applied inside the residual (to the branch), not to x itself.
 x always flows through unchanged as the skip connection.

✓ **OUR CHOICE:** Residual connections around both attention and MLP sub-layers in every transformer block. This is not optional — without them, a 16-layer model would not train. All modern transformers use this structure.

9. Model Depth vs Width — Layers and Dimensions

Given a fixed parameter budget (30M), how should you allocate parameters? Should you make the model wide (large hidden_dim) or deep (many layers)? This tradeoff is one of the most researched questions in LLM architecture.

9.1 The General Wisdom

Scale	Optimal Strategy	Reason
Very large models (10B+)	Wide: larger hidden_dim and intermediate_dim	Scale laws favor width at large compute budgets
Medium models (1B–10B)	Balanced depth and width	Both contribute meaningfully
Small models (< 500M) ✓	Deep and narrow	Depth provides more computational steps per parameter. MobileLLM (Meta 2023) showed this empirically.

9.2 MobileLLM Findings — Critical for Our Scale

Meta's 2023 MobileLLM paper specifically studied sub-500M parameter models and found a counterintuitive result: at small scales, making models deeper (more layers with smaller hidden_dim) consistently outperforms making them wider (fewer layers with larger hidden_dim), at the same parameter count.

MobileLLM Experiment Summary

Config A: 12 layers, hidden=512 → 82M params → benchmark score: X
Config B: 24 layers, hidden=320 → 82M params → benchmark score: X + 3-4%
Config C: 32 layers, hidden=256 → 82M params → benchmark score: X + 2-3%

Deeper + narrower wins at small scale, even with same total parameters.

Why? More layers = more sequential processing steps = more computational depth.
The model can build more complex representations even though each layer is narrower.

9.3 How We Arrived at 16 Layers × 384 Hidden

Working backwards from a 30M parameter budget:

Config	Layers	Hidden	Total Params	Assessment
Option A: Wide, shallow	8	512	~28M	Too few processing steps. Underperforms.
Option B: Medium	12	448	~30M	Acceptable but not leveraging depth advantage.
Option C: Our choice ✓	16	384	~30M	Good depth, reasonable width. MobileLLM-informed.
Option D: Very deep	24	288	~28M	hidden_dim=288 doesn't divide evenly into 6 heads of 64.

✓ **OUR CHOICE:** 16 layers × hidden_dim 384. Leans toward depth over width, consistent with MobileLLM findings at sub-100M scale. head_dim = $384 \div 6$ heads = 64, which is a standard, hardware-efficient value.

9.4 Context Length — 512 Tokens

Context length is the maximum number of tokens the model can process at once. SQL queries plus schema definitions need to fit in this window.

Context Length	Max SQL Complexity	Memory Impact
256 tokens	Simple queries, small schemas	Very small. Might miss complex examples.
512 tokens ✓	Medium queries, typical schemas (3–5 tables)	Small. Covers ~95% of Spider/WikiSQL examples.
1024 tokens	Complex queries, large schemas	4× memory vs 512. Not needed for our data.

✓ **OUR CHOICE:** Context length of 512 tokens. The average Spider example (schema + question + SQL) is around 150–250 tokens. 512 gives comfortable headroom without inflating memory or computation.

10. Complete Architecture Configuration

Bringing everything together: the full hyperparameter specification for our 30M SQL language model.

Hyperparameter	Value	Reason
vocab_size	8,192	SQL-focused vocab. Saves 9.5M params vs 32k vocab.
hidden_dim	384	Core model width. Fits 30M budget with 16 layers.
num_layers	16	Deep-narrow strategy from MobileLLM findings.
num_heads	6	$6 \times 64 = 384$. head_dim=64 is hardware-efficient.
head_dim	64	Standard, efficient. Powers of 2.
intermediate_dim	896	SwiGLU-adjusted $\sim 2.34 \times$ hidden_dim.
context_length	512	Covers 95%+ of SQL examples with room to spare.
positional encoding	RoPE	Relative position awareness, no extra params.
rope_base	10,000	Sufficient for 512 context. Well-tested value.
normalization	RMSNorm	$\sim 15\%$ faster than LayerNorm. Same or better quality.
norm placement	Pre-Norm	More stable training than Post-Norm.
activation	SwiGLU	Best quality at small scale. Gated information flow.
attention type	Multi-Head	Full expressiveness. GQA unnecessary at 30M.
weight tying	Yes	Saves 3.1M params. Input embed = Output LM head.
bias in linear	No	Standard for modern LLMs. Slight speed improvement.
dropout	0.0	Not used during pretraining. May add for finetuning.
total parameters	$\sim 30\text{M}$	Verification: count all weight matrices.

10.1 Parameter Count Verification

Where the 30M Parameters Live

Embedding table (weight-tied): $8192 \times 384 = 3,145,728$ ($\sim 3.1\text{M}$)

Per transformer layer:

Attention Q projection: $384 \times 384 = 147,456$

Attention K projection: $384 \times 384 = 147,456$

Attention V projection: $384 \times 384 = 147,456$

Attention O projection: $384 \times 384 = 147,456$

MLP gate projection: $384 \times 896 = 344,064$

MLP up projection: $384 \times 896 = 344,064$

MLP down projection: $896 \times 384 = 344,064$

Pre-Attn RMSNorm scale: $384 = 384$

Pre-MLP RMSNorm scale: $384 = 384$

Layer total: $= 1,622,784$ (~1.62M)

16 layers: $16 \times 1,622,784 = 25,964,544$ (~26M)

Final RMSNorm scale: $384 = 384$

Grand Total: $3.1M + 26M + \sim 0.4M \approx 29.5M$ ✓

11. Training Configuration

Architecture decisions are only half the story. How you train the model — optimizer, learning rate, schedule, batch size — is equally important and arguably where most training runs fail.

11.1 The Objective Function — Cross-Entropy Loss

Language model training uses next-token prediction. For every token in the input sequence, the model predicts the next token. The loss is the cross-entropy between the predicted probability distribution and the actual next token.

Loss Computation

Input sequence: [<bos>, SELECT, *, FROM, orders, WHERE, amount, >, 100]

Targets (shifted): [SELECT, *, FROM, orders, WHERE, amount, >, 100, <eos>]

For each position i :

predicted = $\text{softmax}(\text{logits}[i])$ (probability over 8192 tokens)

actual = one-hot(target[i]) (1 for the true next token, 0 elsewhere)

loss[i] = $-\log(\text{predicted}[\text{actual token}])$

Total loss = mean of loss[i] across all positions and all sequences in batch.

Perfect model: loss = 0. Random model: loss = $\log(8192) \approx 9.01$.

Good trained model on SQL: expect loss around 1.5–2.5 after training.

11.2 The Optimizer — AdamW

The optimizer is the algorithm that updates model weights based on gradients. AdamW is the near-universal choice for transformer training.

Optimizer	Idea	LLM Usage
SGD	Move in direction of gradient	Not used — too slow to converge on transformers
Adam	Adaptive per-parameter learning rates using momentum and variance estimates	Good but has weight decay issue
AdamW ✓	Adam + correct weight decay (decoupled from gradient update)	Standard for all modern LLMs
Muon (2024)	Momentum + orthogonalization	Very new, promising but untested at scale

AdamW Hyperparameters — The Devil is in the Details

Most failed training runs come from wrong AdamW hyperparameters, not wrong architecture.

Parameter	Our Value	Common Mistake	Why It Matters
learning_rate (peak)	5e-4	Too high (1e-3) or too low (1e-5)	Too high = loss spikes. Too low = slow convergence.
weight_decay	0.1	Default 0.01 or 0	Regularization. 0.1 is standard for LLMs.
beta1	0.9	Default 0.9 (fine)	First moment (gradient momentum). Standard.
beta2	0.95	Default 0.999 (wrong for LLMs)	Second moment. 0.999 is too slow for LLMs — use 0.95.
epsilon	1e-8	Default 1e-8 (fine)	Numerical stability. Rarely needs changing.
grad_clip	1.0	Forgetting it entirely	Clips gradient norm to 1.0. Prevents catastrophic updates.

Critical Note on beta2

The default PyTorch/optax beta2 is 0.999. This was fine for early Adam papers. For LLM training, 0.95 is consistently better — it adapts faster to changing gradient statistics. Using 0.999 does not break training, it just slows it down and can cause instability near learning rate changes. This is one of those hyperparameters that is easy to get wrong by trusting defaults.

11.3 Learning Rate Schedule

The learning rate should not be constant throughout training. Starting high causes instability; staying high too long causes the model to overshoot good minima; ending high prevents convergence.

Schedule	Shape	Advantage	Disadvantage
Constant	Flat line	Simple	Poor convergence
Step decay	Staircase drops	Simple	Abrupt transitions, requires tuning step positions
Cosine decay	Smooth curve from peak to near-zero	Smooth, well-studied	Cannot easily extend training — must restart if you want more steps
WSD (Warmup-Stable-Decay) ✓	Warmup → flat → smooth decay	Can extend training without restart. Easy to continue.	Slightly less explored than cosine

WSD Schedule — Why It Is Better for Us

Standard cosine: LR follows a cosine curve from peak to near-zero over all training steps.
Problem: If you finish training and want to continue, you must restart from the peak LR.

WSD structure:

Phase 1 — Warmup: LR rises from 0 to peak_LR over first ~1,000 steps.

Why: Starting at peak LR causes instability in early steps when weights are random and gradients are large.

Phase 2 — Stable: LR stays at peak_LR for 85% of training.

Why: Most of the learning happens here at full learning rate.

Phase 3 — Decay: LR drops smoothly to ~10% of peak over last 15% of steps.

Why: Fine-tunes the solution found in Phase 2.

Key advantage: If after training you want more tokens, just extend Phase 2 and re-run Phase 3. No restart needed. From MiniCPM paper (2024).

✓ **OUR CHOICE:** WSD schedule: warmup 1,000 steps → stable at $5e-4$ for ~6,600 steps → cosine decay to $5e-5$ over last 1,200 steps. Total ~7,800 steps for 2B tokens at 256k batch size.

11.4 Batch Size and Gradient Accumulation

Batch size in LLM training is measured in tokens, not sequences. A larger token batch means more stable gradient estimates but requires more memory.

Batch Size on TPU v5e-1

Sequence length: 512 tokens

Sequences per batch: 256

Tokens per batch: $256 \times 512 = 131,072$ tokens (~128k)

If we want 256k token batch (more stable): use gradient accumulation of 2.

→ Run 2 forward/backward passes before updating weights.

→ Effective batch = $2 \times 128k = 256k$ tokens.

Total steps at 256k batch, 2B tokens:

$2,000,000,000 \div 262,144 = \sim 7,629$ steps

11.5 Mixed Precision Training

Precision	Bits	Range	Use Case
FP32	32	Large range, high precision	Optimizer states (always)
FP16	16	Limited range — overflow prone	Avoid on TPUs
BF16 ✓	16	Same range as FP32, lower precision	Forward pass + weights on TPU

✓ **OUR CHOICE:** BF16 for model weights and activations. FP32 for optimizer states. TPU v5e natively supports BF16 and it is the standard for all modern TPU training. BF16 has the same exponent range as FP32 (no overflow risk) but half the memory and faster computation.

11.6 Sequence Packing

Naive batching pads all sequences to the same length with padding tokens. This wastes compute on meaningless padding tokens.

Packing vs Padding

Without packing (naive):

Sequence A: 80 tokens + 432 padding tokens

Sequence B: 200 tokens + 312 padding tokens

Useful token ratio: $(80 + 200) / (512 + 512) = 27\%$

You waste 73% of compute on padding.

With packing:

Pack multiple sequences end-to-end until reaching 512 tokens.

Short sequences separated by <eos><bos> boundaries.

Useful token ratio: ~98%+

Packing gives you effectively ~3.7× more useful training signal per compute dollar.

✓ **OUR CHOICE:** Sequence packing. All training examples are packed end-to-end using EOS+BOS boundaries. Attention masking ensures sequences do not attend across document boundaries.

12. Training Data

At 30M parameters, data quality has outsized importance. You cannot compensate for bad data with a clever architecture. The model can only learn what is in the training data.

12.1 Chinchilla Scaling Laws and Our Strategy

The Chinchilla paper (DeepMind, 2022) found the optimal relationship between model size and training tokens for a given compute budget:

Chinchilla Formula

Optimal training tokens $D = 20 \times N$ (where N = number of parameters)

For our 30M parameter model:

Chinchilla optimal $D = 20 \times 30,000,000 = 600,000,000$ tokens (600M)

We are using 2B tokens = 3.3× the Chinchilla optimal.

Why over-train? Chinchilla optimizes for minimum compute to reach a given loss.

It does NOT optimize for best possible model given a fixed model size.

A model trained on more tokens continues to improve — it just becomes less compute-efficient. For inference (using the model), you want the best model, not the most compute-efficient one to train. So over-training is desirable.

12.2 Our 2B Token Data Mix

Source	Tokens	Proportion	Why
SQL files from GitHub (The Stack)	800M	40%	Raw SQL queries, schema definitions, migrations. Pure SQL syntax learning.
StackOverflow SQL Q&A	400M	20%	Real English questions paired with SQL answers. Gold for text-to-SQL task.
SQL documentation (MySQL, PostgreSQL, SQLite)	200M	10%	Teaches function semantics, data types, edge cases.
Python files with embedded SQL	300M	15%	SQL in realistic programming context. Teaches schema relationships.
FineWeb-Edu English text	300M	15%	English language understanding. Model must understand natural language questions.

12.3 Data Quality Over Quantity

A key insight from recent research (FineWeb, DCLM papers): aggressive filtering of data quality beats raw scale. A 100B token high-quality dataset trains better models than a 1T token unfiltered dataset.

- Deduplication: Remove exact and near-duplicate SQL files. Same schema repeated 1,000 times teaches nothing new.
- Length filtering: Remove files under 50 characters (no useful signal) and over 8,000 characters (probably auto-generated junk).
- Quality heuristics for SQL: Remove files with > 50% commented-out code, files that are just schema dumps with no queries, files that do not contain any SQL keywords.
- Shuffle: Randomize the order of training examples. Correlated order creates instability.

12.4 Finetuning Data — Text-to-SQL Task

After pretraining on raw SQL + text, we finetune specifically on text-to-SQL examples to align the model to the actual task.

Dataset	Examples	Quality	Use
sql-create-context	78k	High	Core finetuning. Schema + question → SQL format.
Spider train set	7k	Very high	Gold standard benchmark. Complex multi-table queries.
BIRD train set	9k	Very high	Harder realistic queries. External knowledge required.
gretelai/synthetic_text_to_sql	100k	Medium (synthetic)	Diversity. Covers edge cases not in Spider.
WikiSQL	61k	Medium (simple)	Volume. Simple single-table queries.

12.5 Finetuning Input Format

Consistent formatting during finetuning is critical. The model learns the format as part of the task. Inconsistent delimiters confuse small models significantly more than large ones.

Standard Finetuning Format

<schema>

```
CREATE TABLE employees (id INT PRIMARY KEY, name VARCHAR(100), salary INT, dept_id INT);
```

```
CREATE TABLE departments (id INT PRIMARY KEY, dept_name VARCHAR(100));
```

</schema>

<question>

What is the average salary in each department?

</question>

<sql>

SELECT d.dept_name, AVG(e.salary) as avg_salary

FROM employees e

JOIN departments d ON e.dept_id = d.id

GROUP BY d.id, d.dept_name;

</sql>

Use exactly these delimiters every time. At 30M params, consistency matters more than at larger scales where the model can figure out variations.

13. Evaluation — How to Measure Success

For SQL models, you have a major advantage over general language model evaluation: you can execute the output and check if it is correct. This gives you a much stronger signal than perplexity or n-gram matching.

13.1 Metrics Hierarchy

Metric	What It Measures	Strength
Execution Accuracy (EX)	Does running the SQL on the database produce the correct result set?	★★★★★ Gold standard. Multiple valid SQL queries can produce the same result.
Exact Match (EM)	Is the generated SQL identical to the reference SQL?	★★★ Flawed — many valid SQLs exist for one question. Understates true accuracy.
Valid SQL Rate	Is the generated SQL syntactically valid?	★★ Necessary but not sufficient. Easy to game.
Test-Suite Accuracy	Does SQL pass a suite of test inputs, not just one?	★★★★★ More robust than single-example execution.

✓ **OUR CHOICE:** Execution Accuracy (EX) as primary metric. Build an evaluation harness that runs generated SQL against SQLite databases and compares result sets.

13.2 Standard Benchmarks

Benchmark	Size	Difficulty	Target Score (30M model)
WikiSQL	80k test	Easy (single table only)	60–70% EX
Spider (Easy)	~500 examples	Medium	45–55% EX
Spider (Full)	~1,034 test examples	Hard	30–42% EX
BIRD (Easy)	~1,500 examples	Very hard	25–35% EX

13.3 Diagnosing Failure Modes

After evaluation, categorize failures to know where to focus improvement:

- Schema linking errors: Model refers to wrong column name or table. Fix: more StackOverflow SQL data with diverse schema names.
- Aggregation errors: Uses SUM instead of COUNT, or forgets GROUP BY. Fix: targeted finetuning examples.
- JOIN direction errors: Wrong ON condition or wrong JOIN type. Fix: more multi-table examples with explicit annotations.
- Syntax errors: Generated SQL cannot even parse. Fix: more raw SQL pretraining.
- Hallucinated columns: References columns that do not exist in schema. Fix: stricter attention to schema tokens.

14. JAX and TPU — Why This Stack

We chose JAX + Flax + Optax running on Google's TPU v5e-1. This section explains each choice and what it means in practice.

14.1 Why JAX Over PyTorch?

Aspect	PyTorch	JAX
Primary design target	GPUs (CUDA ecosystem)	TPUs + GPUs equally
Execution model	Eager by default (runs immediately)	Lazy + compiled (traces, then runs fast)
TPU support	Via torch_xla — works but some friction	Native — TPU is a first-class target
Functional style	Optional (mostly mutable state)	Required (all pure functions)
Speed on TPU	Good	Excellent — designed for it
Learning curve	Gentle	Steeper — functional paradigm is different

✓ **OUR CHOICE:** JAX because our hardware is TPU v5e-1. JAX compiles to XLA which is TPU's native instruction set. The functional programming model is more work to learn but produces faster, more predictable code on TPUs.

14.2 The JAX Programming Model — Key Concepts

Pure Functions

JAX requires all functions to be pure: same input always produces same output, no side effects. This allows JAX to trace the function and compile it.

Pure Function Requirement

```
# Wrong — has side effect (prints)
def bad_forward(params, x):
    print('running forward') # side effect!
    return model.apply(params, x)

# Right — pure function
def good_forward(params, x):
    return model.apply(params, x)
```

jit — Just-In-Time Compilation

The most important JAX transformation. Decorating a function with `@jax.jit` compiles it to XLA on first call. Every subsequent call runs the compiled version — much faster.

jit Usage

```
@jax.jit
def train_step(state, batch):
    loss, grads = jax.value_and_grad(loss_fn)(state.params, batch)
    state = state.apply_gradients(grads=grads)
    return state, loss

# First call: ~30-60 seconds to compile
# Every subsequent call: milliseconds
# CRITICAL: Always jit your train_step — never run it un-jitted in a loop
```

value_and_grad — Computing Loss and Gradients Together

JAX computes the gradient of any scalar-valued function using automatic differentiation. `value_and_grad` returns both the function output and its gradient in one pass — efficient.

lax.scan — Efficient Loops

For the 16 transformer layers, never use a Python for loop inside jit. Python loops unroll at compile time, causing very slow compilation and large memory use. Use `jax.lax.scan` instead.

Loop vs scan

```
# Wrong — Python loop inside jit (compiles 16 copies of the layer)
for i in range(16):
    x = transformer_block(x, params[i])

# Right — lax.scan (compiles once, scans over stacked params)
def scan_fn(x, layer_params):
    return transformer_block(x, layer_params), None

x, _ = jax.lax.scan(scan_fn, x, stacked_layer_params)
```

14.3 Flax — Neural Network Library

Flax is JAX's most mature neural network library. It provides the `nn.Module` pattern for defining models, similar to PyTorch's `nn.Module` but functional.

Key difference from PyTorch: Flax modules have no persistent state. Parameters are passed in explicitly and returned explicitly. The module itself is just a template that defines the computation.

14.4 Optax — Optimizer Library

Optax provides all standard optimizers (AdamW, SGD, etc.) and learning rate schedules as composable transformations. It is the standard optimizer library for JAX.

14.5 TPU v5e-1 Specifics

TPU v5e-1 Specs (Your Hardware)

BF16 TFLOPS: 197

HBM Memory: 16 GB

Memory Bandwidth: 819 GB/s

Practical efficiency: ~40–50% (expect ~80–100 usable TFLOPS)

Memory fit for our model:

Weights (BF16): $30\text{M} \times 2 \text{ bytes} = 60 \text{ MB}$

Optimizer states: $30\text{M} \times 8 \text{ bytes} = 240 \text{ MB}$ (FP32 Adam moments)

Activations: ~200–400 MB depending on batch

Total: ~700 MB–1 GB

Available: 16,000 MB

Headroom: ~15 GB — comfortable, room for large batches

Training time for 2B tokens:

FLOPs needed: $6 \times 30\text{M} \times 2\text{B} = 3.6 \times 10^{17}$

At 100 eff. TFLOPS = 10^{14} FLOPS/sec

Time: $3.6 \times 10^{17} / 10^{14} = 3,600 \text{ seconds} \approx 1\text{--}2 \text{ hours}$

15. The Complete Pipeline — End to End

Everything together, in order, from raw data to evaluated model.

Phase	Step	What Happens	Output
1. Data	Download + Filter	The Stack SQL files, StackOverflow dump, FineWeb-Edu slice, SQL docs	~8B tokens raw
1. Data	Deduplicate	MinHash near-dedup, exact dedup. Remove repetitive files	~3B tokens clean
1. Data	Sample to budget	Randomly sample to hit 2B token mix (40/20/10/15/15%)	2B tokens final
2. Tokenizer	Train BPE	Train 8,192 vocab BPE on 100M char SQL+English sample	tokenizer.json
2. Tokenizer	Tokenize corpus	Convert all 2B tokens of text to integer token IDs	token arrays (.npy)
3. Pretrain	Initialize model	Create 30M param model with std=0.02 weight init	random weights
3. Pretrain	Train loop	~7,800 steps, WSD schedule, AdamW, 256k token batch	pretrained weights
3. Pretrain	Monitor	Watch loss curve, grad norm, eval every 500 steps	training logs
4. Finetune	Format task data	Format Spider/WikiSQL/synthetic as schema+question→SQL	255k examples
4. Finetune	Finetune	3–5 epochs, LR=5e-5, ~30–45 minutes on TPU	finetuned weights
5. Evaluate	WikiSQL eval	Run execution accuracy on WikiSQL test set	~60–70% EX
5. Evaluate	Spider eval	Run execution accuracy on Spider test set	~30–42% EX
5. Evaluate	Error analysis	Categorize failures by type	Improvement targets
6. Iterate	Fix weak spots	Generate synthetic data for failure modes, re-finetune	Better weights

15.1 Recommended Week-by-Week Schedule

Week	Focus	Milestone
Week 1	Environment + data	JAX TPU working. Data downloaded. Tokenizer trained.
Week 2	Model implementation	30M model forward pass working. Parameter count verified.

Week	Focus	Milestone
Week 3	First training run	Train on 500M token subset. Loss curve looks healthy.
Week 4	Full pretraining + finetune	2B token pretrain complete. Finetuned on SQL task.
Week 5	Evaluation + iteration	Spider EX measured. Failure modes identified. Re-finetune.
Week 6+	Optimization	Improve data mix, try curriculum learning, push Spider score.

16. Quick Reference — All Decisions at a Glance

A single-page summary of every architectural decision, the options we considered, and our final choice.

Decision	Options Considered	Our Choice + Reason
Tokenization	Word-level, Character-level, BPE, SentencePiece	BPE — handles any text, efficient subword compression
Vocabulary size	32k, 16k, 8k, 4k	8k — SQL domain only needs small vocab, saves 9.5M params
Positional encoding	Learned absolute, Sinusoidal, ALiBi, RoPE	RoPE — relative position awareness, no extra params
RoPE base	10k, 100k, 500k	10k — sufficient for 512 context length
Normalization type	LayerNorm, RMSNorm, GroupNorm	RMSNorm — 15% faster, same or better quality
Norm placement	Pre-Norm, Post-Norm	Pre-Norm — more stable training
Activation function	ReLU, GELU, SwiGLU	SwiGLU — best quality, gated information flow
Attention variant	MHA, GQA, MQA	MHA — full expressiveness, GQA overkill at 30M
Flash Attention	Yes, No	Yes — free speedup, same result
Weight tying	Tied, Separate	Tied — saves 3.1M params, no downside
Depth vs Width	Wide+shallow, Balanced, Deep+narrow	Deep+narrow — MobileLLM finding at sub-100M scale
Hidden dim	256, 384, 512	384 — fits 30M budget with 16 layers
Num layers	8, 12, 16, 24	16 — good depth for SQL task reasoning
Context length	256, 512, 1024	512 — covers 95%+ of SQL examples
Optimizer	SGD, Adam, AdamW, Muon	AdamW — universal LLM standard
LR schedule	Constant, Cosine, WSD	WSD — can extend training without restart
Beta2	0.999 (default), 0.95	0.95 — adapts faster, better for LLMs
Batch size	64k, 128k, 256k tokens	256k tokens — stable gradient estimates
Precision	FP32, FP16, BF16	BF16 for compute, FP32 for optimizer states
Sequence handling	Padding, Packing	Packing — ~98% compute efficiency vs ~27%
Framework	PyTorch, JAX, TensorFlow	JAX — native TPU support, compiled execution
NN library	Raw JAX, Flax, Equinox	Flax — mature, well-documented, Google-supported